LAB 5

STATIC CODE ANALYSIS REPORT

E-commerce Cart System | Python

Name: Sachin S
SRN: PES2UG24CS422

Section -G

1. Detailed Analyzer Findings

1.1 Pylint

Reported score: **9.69/10**. Pylint reported the following two messages:

Code	Reported location	Meaning	Recommended action
C0103	Line 20, column 0	Constant name `total_sales` does not follow UPPER_CASE naming style.	Rename to `TOTAL_SALES` if it is truly a module constant, or remove/restructure the global state if it is not needed.
W0603	Line 31, column 4	Function uses the `global` statement.	Avoid global mutable state. Derive the cart total from cart contents or encapsulate state in a cart object.

1.2 Bandit

Bandit reported: **No issues identified.** The scan covered 98 lines of code. Reported issue counts were zero for undefined, low, medium and high severity, and zero for all confidence levels.

The uploaded text file also contains a PowerShell NativeCommandError diagnostic around the command invocation. The actual Bandit scan output follows it and reports zero findings. This wrapper diagnostic is not a Bandit vulnerability finding.

1.2 Flake8

Flake8 reported one PEP 8 line-length violation: `ecommerce_cart_system.py:37:80:`

```
E501 line too long (83 > 79 characters)
```

Recommended correction: split the long expression or function call over multiple lines so that each line is no longer than 79 characters. Rerun Flake8 after editing to confirm that E501 disappears.

2. Issues Addressed in the Revised Program

The revised program prepared for the submission applies the following changes to the originally uploaded source. These are code changes made during the preparation of the corrected program; they are distinct from the analyzer messages shown above.

Original issue	Change made	Benefit

Mutable default argument	Changed audit log default from an empty list to `None`; only append when a list is supplied.	Prevents state leaking between function calls.
Bare exception handling	Replaced broad exception swallowing with an explicit membership check and Boolean result.	Avoids hiding unexpected errors.
Dynamic `eval()`	Replaced expression evaluation with a validated numeric discount rate from 0 to 1.	Prevents arbitrary Python expression execution.
MD5 hashing	Replaced MD5 with SHA-256.	Uses a stronger modern hash algorithm.
Predictable PIN generation	Replaced `random` with `secrets`.	Uses a cryptographically secure random source.
Shell command execution	Removed `os.system()`; logs receipt path through Python logging.	Avoids shell injection risk from filenames.
Hard-coded payment secret	Removed credential-like value and its print statement.	Avoids exposing secrets in source/output.
Manual file open/close	Used `Path.write_text()` with UTF-8 encoding.	Simplifies file handling.
Input validation gaps	Validated item ID, name, price, quantity and discount rate.	Rejects invalid values early.

3. Functional Verification

The revised Python program was syntax-checked and exercised with targeted functional/security checks during preparation. The checks recorded as passed were: [PASS] Python compilation / syntax check

[PASS] Cart total and quantity summary calculations

[PASS] 10% discount calculation

[PASS] SHA-256 digest generation

[PASS] Four-digit checkout PIN format

[PASS] Audit logging and add/remove cart operations

[PASS] Rejection of a malicious discount expression

Note: these are functional checks, not substitutes for the three static analyzers. The analyzer results are reported separately in Section 2.

4. Reflection and Conclusion

What static analysis contributes: Static analysis checks source code without running the application. It helps detect coding mistakes, unsafe patterns and style problems early in development.

What the three tools contributed: Pylint assessed code quality and reported naming/global-state messages; Bandit checked for common Python security issues and reported none; Flake8 checked formatting and found one line-length violation.

What was learned: Code can appear to work while still containing security and maintainability risks. Using multiple tools provides complementary checks. Findings should be reviewed, fixed in context and verified again against the final source file.

Conclusion: The submitted scan results show Pylint 9.69/10, zero Bandit findings and one Flake8 E501 violation. The revised program also addresses several important security and reliability weaknesses from the original source. For a fully

consistent final record, rerun all three tools against the exact final revised file and replace the earlier reports if the results change.